

Rust MAVLink Mission Companion

A scrappy lab-style plan for building a drone mission control portfolio project

What: Rust + MAVLink + ArduPilot SITL, end to end

Why: DefTech job apps want this exact stack

Time: 2-4 weeks, ~4 labs

Level: you already know Rust, this fills the drone-specific gaps

Why this exists

You're a strong Rust engineer (Reth ExEx, LMDB, network programming) looking at DefTech job posts that all want MAVLink + ArduPilot + 'kinematics and trajectory planning'. None of that is in your current stack.

This isn't a course - it's a build plan dressed up like a lab manual, because that structure (goal, steps, acceptance criteria, questions you should be able to answer) is genuinely a good forcing function for finishing a side project instead of it dying after week one.

Each lab ends with stuff you can point to on a CV or talk through in an interview, not just 'I followed a tutorial'.

Put together: June 2026

Contents

(Word/LibreOffice needs to refresh this field on open - select all + F9, or just say yes if it asks to update fields.)

The idea

What you're actually building

A standalone Rust service - call it Mission Companion - that talks to ArduPilot (SITL first, real hardware maybe later), pulls in live telemetry over MAVLink, plans and flies a simple mission with basic motion kinematics baked in, and shows you what's happening in a browser while it does it.

This is the same pattern real drones use: a 'companion computer' sitting next to the flight controller, handling the smart stuff, while the flight controller handles low-level stabilization. You're not reinventing ArduPilot, you're building the brain that talks to it.

How the pieces fit together

Five moving parts, roughly in this order:

#	Piece	What it does
1	ArduPilot SITL (fake drone)	Simulated aircraft. Talks MAVLink over UDP, no hardware risk.
2	Transport / Telemetry module	Reads the MAVLink stream, turns raw messages into things you can actually use.
3	Mission Planner	Builds the flight plan and works out a kinematically sane path between waypoints.
4	Executor	Keeps the plan in sync with reality, sends the next command back to SITL.
5	Web dashboard	Shows you the drone moving around live, because staring at log lines gets old fast.

Data flows: SITL <-> Telemetry -> Mission Planner -> Executor -> SITL (commands), and Telemetry -> Dashboard on the side for visualization.

Stack

Piece	Tech	Why
Core service	Rust, async/await (tokio)	Where all the logic lives
Telemetry/control protocol	MAVLink v2 (rust-mavlink crate)	How you talk to the autopilot
Autopilot simulator	ArduPilot SITL (ArduCopter)	Fake drone, zero hardware risk
Containers	Docker / docker-compose	So SITL setup is reproducible, not a snowflake
Frontend	Next.js / React (Leaflet or similar for the map)	Live view of the mission

Piece	Tech	Why
Tests	cargo test, proptest for the kinematics	Prove the math doesn't lie
Hardware (later, optional)	Your RadioMaster TX over ELRS/CRSF	Manual override path, nice future extension

How to keep yourself honest each week

After each lab, write a short README update. Doesn't need to be fancy - just:

- 3-5 sentences on what you built
- Why you made the calls you made (the 'why' matters more than the 'what' in interviews)
- A screenshot or log snippet showing it actually works
- Quick answers to that lab's questions (these are the ones an interviewer will actually ask)
- An honest list of what's still broken or fake

Tag or branch each lab separately (lab-1, lab-2...) if you're using git - it's a small thing but it shows you can structure work, and that reads well to anyone skimming your repo.

LAB 1

Get SITL running and talk to it from Rust

Week 1 - roughly 12-16 hours

Goal

Get ArduPilot SITL running, then write a Rust client that connects over MAVLink, listens to the telemetry stream, and actually understands what it's receiving.

What you'll actually build

1. Run ArduPilot SITL (ArduCopter) in Docker, with MAVLink reachable over UDP from outside the container.
2. Hook up QGroundControl or Mission Planner first, just to sanity-check SITL works before you write any code.
3. Start a Rust project, add the mavlink crate (rust-mavlink), connect over UDP.
4. Parse HEARTBEAT, ATTITUDE, GLOBAL_POSITION_INT, and SYS_STATUS into your own clean structs - not raw MAVLink types passed around everywhere.
5. Add structured logging (tracing) so you can actually see what's happening, with a log file you could dig through later.
6. Write a handful of unit tests for the unit-conversion functions (lat/lon, angles) - it's the cheapest place to catch off-by-one-style bugs.

Background you'll want

MAVLink is a small binary message protocol built for bandwidth-constrained links (telemetry radios, satellite, you name it), and it's the de facto standard in drones. Messages are defined in XML dialects (common.xml, ardupilotmega.xml), each with an ID, a fixed set of fields, and a checksum. MAVLink v2 adds extended message IDs and optional signing over v1 - for anything modern, you want v2.

In the real world MAVLink rides over UART (the physical telemetry radio) or over UDP/TCP when you're talking to a companion computer or a simulator. SITL gives you the full ArduPilot firmware running against a simulated physics model instead of real sensors and motors - same MAVLink interface as the real thing, zero chance of breaking a propeller.

Steps

1. Install Docker if you haven't already, sanity check with ``docker run hello-world``.
2. Spin up ArduPilot SITL in a container (ardupilot/ardupilot-sitl image, or build your own following ardupilot.org/dev). Confirm it's broadcasting MAVLink on UDP port 14550 (the standard GCS port).
3. Connect QGroundControl (or Mission Planner under Wine) to it and check the drone shows up on the map with a sane-looking HUD.
4. ``cargo new mission-companion``, add mavlink, tokio, tracing, tracing-subscriber, anyhow.
5. Connect to the SITL MAVLink stream over UDP as a second GCS client (SITL is fine with multiple simultaneous connections).
6. Write an event loop that reads incoming MAVLink messages and dispatches on the message type.

7. For each of the four target message types, build your own struct with human-readable values - degrees instead of raw $\text{int32} * 1e7$ coordinates, meters for altitude, radians and degrees for attitude.
8. Wire up tracing: TRACE for every raw message, INFO for state changes (mode switch, arm/disarm), WARN if heartbeat goes missing for more than 3 seconds.
9. Unit test the conversion functions - negative coordinates, edge values, the usual suspects.

One thing that'll save you time

The rust-mavlink crate generates types from XML dialects at build time via build.rs. Make sure the 'ardupilotmega' dialect is enabled, not just 'common' - otherwise a bunch of ArduPilot-specific messages just won't show up, and you'll waste an hour wondering why.

Done when...

- SITL comes up with one command (docker-compose up or similar), no manual config tweaking.
- Your Rust client holds a connection for 5+ minutes without crashing or leaking memory (just eyeball RSS if you're not sure).
- Logs show readable telemetry - degrees and meters, not raw bytes or magic numbers.
- All the conversion functions have tests, `cargo test` is green.
- `cargo clippy` doesn't complain (or you've got a good reason for the #[allow] you added).

Questions you should be able to answer afterward

1. What's actually different between MAVLink v1 and v2, and why does it matter for a modern ArduPilot setup?
2. Why does SITL talk UDP instead of emulating UART directly, and what does that mean for your code once it eventually talks to a real flight controller over an actual serial port?
3. Why does a missing HEARTBEAT matter so much for a companion system, practically speaking?
4. Why decode raw MAVLink fields right at the boundary instead of passing them around raw and converting later?

LAB 2

Take control: commands, kinematics, and a path planner

Week 2 - roughly 16-20 hours

Goal

Send real flight commands to get the drone flying under your control, then build a trajectory planner that respects basic motion limits instead of just teleporting setpoints around.

What you'll actually build

7. Arm the drone, switch to GUIDED, take off - via `COMMAND_LONG` / `SET_MODE`, with actual confirmation from telemetry, not fire-and-hope.
8. Send position targets via `SET_POSITION_TARGET_LOCAL_NED` (or `_GLOBAL_INT`) in GUIDED mode.
9. Build a simple point-mass kinematic model: max velocity, max acceleration, max yaw rate.
10. Write a trajectory generator between arbitrary waypoints that respects those limits - trapezoidal velocity profile is the minimum bar; cubic splines or a Dubins path are the stretch goal.
11. Build an Executor that drives the mission: sends the next setpoint, checks it against real telemetry, advances to the next waypoint.
12. Unit and property-based tests for the kinematic model and the path generator.

Background you'll want

GUIDED mode lets a companion computer steer the drone in real time by sending position/velocity targets, instead of uploading a fixed mission upfront like AUTO mode does. It's the right mode whenever the plan might change based on live data.

Point-mass kinematics just means: limit velocity (first derivative of position), acceleration (second derivative), and for non-circular paths, the turn radius implied by max bank angle (multirotor) or minimum turn radius (fixed-wing). A trapezoidal velocity profile is the simplest version - accelerate at constant rate, cruise at max speed, decelerate at constant rate.

Dubins paths solve for the shortest path between two oriented points given a minimum turn radius - mostly relevant for fixed-wing, and a simplification for multirotors, but it's exactly the kind of thing job posts mean when they say 'trajectory planning algorithms', so it's worth knowing even if your MVP doesn't need it.

Steps

10. Write `arm_and_set_mode()`: check EKF is happy from telemetry, send `COMMAND_LONG` with `MAV_CMD_COMPONENT_ARM_DISARM`, switch to GUIDED via `SET_MODE`, and actually wait for confirmation from telemetry/ACK instead of assuming it worked.
11. Send `MAV_CMD_NAV_TAKEOFF` to a target altitude, wait until telemetry shows you've reached it (say, within $\pm 0.5\text{m}$).
12. Design a Waypoint struct (coordinates, approach speed, arrival tolerance) and a Mission as an ordered list of them.
13. Build a TrapezoidalProfile: given distance, max velocity, max acceleration, compute position/velocity over time.

14. Generate setpoints along the straight line between waypoints at a rate the autopilot expects (5-10 Hz is typical for SET_POSITION_TARGET_LOCAL_NED).
15. Build the Executor as an async tokio task: read current telemetry -> compare against the active waypoint -> advance if within tolerance, otherwise compute and send the next setpoint from the kinematic model.
16. Unit test TrapezoidalProfile: never exceeds the configured max velocity/acceleration, final position matches target exactly, symmetric accel/decel when limits match.
17. Property-based test (proptest) for the path generator: for arbitrary (reasonable) waypoint pairs and limits, the trajectory always moves monotonically toward the goal and ends exactly there.

Easy mistake to avoid

If you send setpoints slower than roughly 2 Hz, ArduPilot treats GUIDED commands as stale and falls back to holding position (a failsafe). Make sure the Executor keeps a steady update rate no matter what else is going on - it's also a good way to show you understand real-time-ish guarantees in async Rust, not just 'async because it's trendy'.

Done when...

- The drone flies a 4+ waypoint mission in SITL without dropping out of GUIDED or tripping a failsafe.
- Tests prove velocity and acceleration never exceed your configured limits at any point in the simulation.
- Missions are defined in a config file (TOML/YAML/JSON), not hardcoded - no recompiling to fly a different route.
- Property tests run reliably across 256+ random cases without failing.

Questions you should be able to answer afterward

5. What's the real difference between AUTO and GUIDED in terms of who's responsible for building the trajectory - the autopilot or your companion code?
6. Why is a trapezoidal velocity profile only an approximation, and when would a real multirotor need an S-curve (jerk-limited) profile instead?
7. What goes wrong if the Executor sends setpoints at a jittery rate, and how does tokio help you avoid that?
8. Why is trajectory planning genuinely simpler for a multirotor than for fixed-wing (Dubins path), physically speaking?

LAB 3

Clean up the architecture and make it not fragile

Week 3 - roughly 14-18 hours

Goal

Turn the working prototype into something with clear module boundaries, proper async wiring, real error handling, and tests that don't depend on you remembering to run them manually.

What you'll actually build

13. Split into clear modules - transport (MAVLink I/O), telemetry (current state), mission (planning, pure logic), executor (orchestration) - connected via channels instead of direct calls.
14. Replace String errors and panics with a real error enum (thiserror).
15. Handle graceful shutdown: Ctrl+C or a lost MAVLink connection should leave the drone in a sane state, not just kill the process.
16. Add integration tests that spin up a mock (or real) MAVLink server and exercise the system from the outside.
17. Basic CI (GitHub Actions): build, test, clippy, fmt --check on every push.
18. Write 3 short ADRs (Architecture Decision Records) documenting decisions you actually made.

Background you'll want

In systems that control physical hardware, it really matters to separate the 'dangerous' I/O layer (network reads, dropped connections, corrupted packets, timeouts) from the 'trusted' domain layer (planning logic working on already-validated data). Tokio channels let you do that split without shared mutable state or locks, which also makes each piece independently testable.

Graceful shutdown means: close connections cleanly, let in-flight tasks finish, don't leave the drone mid-trajectory with no more setpoints coming. If your companion process just dies, the autopilot stops getting updates and falls into failsafe behavior - this isn't a nice-to-have, it's a safety property.

An ADR is a short writeup (often under a page) of one decision: context, alternatives considered, what you picked, and the consequences. It's a normal practice on real engineering teams, and having a few in your portfolio repo signals you can work like part of a team, not just solo-code in a vacuum.

Steps

18. Pull out transport::mavlink_link - wraps the UDP/MAVLink details, exposes an async send(command) and a telemetry event stream, nothing else.
19. Pull out telemetry::state - keeps the latest snapshot (position, velocity, mode, EKF status) in a tokio::sync::watch, readable by anything without blocking.
20. Pull out mission - pure structs and planning functions from Lab 2, with zero MAVLink dependency. It should compile and test fine on its own, which is the real proof your domain logic isn't secretly coupled to the transport.
21. Pull out executor - the only module that knows about all the others, wiring them together.

22. Add a `MissionError` enum (thiserror) with variants like `ConnectionLost`, `ArmRejected`, `ModeChangeTimeout`, `WaypointUnreachable`. Remove `unwrap()/expect()` from anything that depends on the network or telemetry.
23. Handle `tokio::signal::ctrl_c` with a clean cancel of all tasks (`CancellationToken` from `tokio-util`), optionally triggering RTL (return-to-launch) before exiting.
24. Write an integration test under `tests/` that spins up SITL (or a lightweight mock MAVLink server), runs Mission Companion against it, and checks a 3-waypoint mission completes in a reasonable time.
25. Set up `.github/workflows/ci.yml`: build, test, clippy, fmt.
26. Write 3 ADRs - e.g. 'channels vs `Arc<Mutex<State>>`', 'why GUIDED and not AUTO for v1', 'why trapezoidal profile for the MVP' - in `docs/adr/`.

Done when...

- No `unwrap()/expect()/panic!` on any path that depends on external input or the network (tests and genuine invariants are fine).
- The mission module compiles and tests with zero MAVLink/network dependency.
- CI is green on a clean clone of the repo.
- Ctrl+C mid-mission leaves the drone in a known state - show the before/after log in your README.
- At least 3 ADRs exist.

Questions you should be able to answer afterward

9. Why does decoupling domain logic (mission) from infrastructure (MAVLink I/O) matter even more for systems that control physical hardware?
10. What's the actual advantage of `tokio::sync::watch` over `Arc<Mutex<T>>` for a telemetry snapshot read by multiple tasks?
11. Why is just killing the process unacceptable for something controlling an aircraft mid-flight, concretely?
12. What's the tradeoff of `CancellationToken` versus just dropping a channel to signal shutdown?

LAB 4

Dashboard, demo, and making it presentable

Week 4 - roughly 12-16 hours

Goal

Build a live web dashboard for the mission, and get the whole project into a state you'd actually be comfortable linking from a CV or walking through in an interview.

What you'll actually build

19. WebSocket endpoint on the Rust side streaming telemetry + mission state as JSON.
20. Simple REST endpoints to start/stop a mission and upload waypoint config.
21. A frontend (Next.js/React) with a live map (Leaflet or similar), showing the flight path, current waypoint, and system state (armed/guided/mode).
22. A short screen-recorded demo (2-4 min) of the full loop: SITL up -> Mission Companion up -> mission loaded -> watch it fly on the dashboard.
23. A real README: overview, architecture diagram, quick start that actually works for a stranger, and an honest 'known limitations' section.
24. A short interview-prep doc (1-2 pages) - what you built, what tradeoffs you made, what you'd do differently with more time.

Background you'll want

WebSockets let the server push updates instead of the client polling - which matters here because at 5-10 Hz, HTTP polling would mean a lot of wasted requests and added latency for no good reason.

In DefTech teams (and most real engineering teams, honestly), documentation quality and 'does this actually run on a clean machine' often matter more to how a candidate is perceived than how clever the algorithm is - it's a direct signal of how you'd work day to day.

Steps

27. Add axum (or actix-web). Build a WebSocket handler that reads telemetry::state (the watch channel) and pushes JSON snapshots at a fixed rate (5 Hz works fine), independent of the underlying MAVLink message rate.
28. Add REST endpoints: POST /mission (upload waypoints as JSON), POST /mission/start, POST /mission/abort (switch to RTL/LOITER).
29. Build a Next.js app (frontend/ in the same repo is fine), wire up Leaflet (or deck.gl if you want to flex on geospatial viz), show current position, the flown track, and waypoint markers.
30. Add a status panel: flight mode, armed/disarmed, altitude, speed, distance to next waypoint, GPS/EKF status.
31. Run the full end-to-end flow a few times, make sure it doesn't desync if the WebSocket drops and reconnects mid-mission.
32. Record the demo with any screen recorder, keep it tight, upload unlisted on YouTube or just drop the file in the repo if it's small enough.

33. Write the final README: overview -> architecture diagram -> quick start -> what each lab added -> known limitations -> roadmap.
34. Write the interview-prep doc - short, meant for talking through out loud, not for re-reading the README.

Done when... (final bar for the whole project)

- Anyone with Docker and Node can bring up the whole stack (SITL + Rust service + frontend) from the README with no extra help from you.
- The dashboard shows the flight live, smoothly enough to actually watch.
- The demo video shows the full loop with no suspicious cuts hiding a failure.
- The README and prep doc are written for a tech lead skimming for 10 minutes before an interview, not for you to admire later.
- The 'Known Limitations' section is specific and honest - e.g. 'no wind modeling', 'no obstacle avoidance', 'straight-line segments only, no Dubins path yet'.

Questions you should be able to answer afterward

13. Why is push (WebSocket) the right model here instead of pull (HTTP polling), and when would that not hold?
14. Which of your 'Known Limitations' is most likely to get poked at in an interview, and what's your short answer to 'how would you fix that with more time'?
15. How would this architecture (companion computer + MAVLink) differ from one where you're integrating directly with low-level flight controller firmware, say on an STM32?

How this maps back to the job post

Direct line from each lab to the specific things DefTech/Rust job posts ask for - the same words you'll see in requirements sections.

Job requirement	Where it's covered
3+ years commercial Rust	The whole thing - production-grade async architecture throughout
Async, multithreading	Lab 1 (event loop), Lab 3 (channels, graceful shutdown)
Motion kinematics / basic physics	Lab 2 (trapezoidal profile, acceleration limits)
Trajectory planning algorithms	Lab 2 (waypoint trajectory generator, Dubins path concept)
MAVLink (nice-to-have)	Labs 1, 2, 3 - used throughout
ArduPilot (nice-to-have)	Lab 1 - SITL as the main dev environment
Telemetry (nice-to-have)	Lab 1 (ingestion), Lab 4 (live visualization)
DB/API/message broker integration	Lab 3 (REST/WebSocket API), Lab 4 (mission REST endpoints)

If you want to keep going after week 4

- Hook up the actual RadioMaster TX over ELRS/CRSF as a manual override channel - shows you understand how manual and autonomous control interact.
- Swap the straight-line segments for a real Dubins path or spline interpolation - directly covers the 'trajectory planning algorithms' nice-to-have at a deeper level.
- Add the simplest possible obstacle avoidance (a static no-fly polygon, say) - closer to what real DefTech production work actually looks like.
- Move part of this onto a real Pixhawk-compatible board if you end up getting one - proves you're not purely simulation-only.

That's the plan. Good luck, and don't let week 3 (the boring refactor week) be the one you skip - it's the one that actually reads as senior.